

CS308 — Software Engineering

# Aktour ViaBalkan

## Architecture Design Documentation

Design Patterns · Architecture Style · Communication & Security

---

### Project Team

Furkan Duman · Dogukan Yurtturk · Eren Çapar · Can Aysen

International University of Sarajevo

May 2025 · Version 1.0

## Contents

---

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>System Overview</b>                                       | <b>2</b>  |
| <b>2</b> | <b>Architecture Style</b>                                    | <b>2</b>  |
| 2.1      | Selected Style: Layered Client-Server Architecture . . . . . | 2         |
| 2.1.1    | Why Client-Server? . . . . .                                 | 2         |
| 2.1.2    | Why Layered (N-Tier)? . . . . .                              | 2         |
| 2.1.3    | Why NOT Microservices? . . . . .                             | 3         |
| 2.1.4    | Why NOT Event-Driven? . . . . .                              | 3         |
| <b>3</b> | <b>Architecture Diagram</b>                                  | <b>3</b>  |
| 3.1      | High-Level System Diagram . . . . .                          | 3         |
| 3.2      | Data Model Relationships . . . . .                           | 4         |
| <b>4</b> | <b>Design Patterns</b>                                       | <b>4</b>  |
| 4.1      | Repository Pattern . . . . .                                 | 4         |
| 4.2      | Data Transfer Object (DTO) Pattern . . . . .                 | 5         |
| 4.3      | Mapper Pattern — MapStruct . . . . .                         | 5         |
| 4.4      | Singleton Pattern . . . . .                                  | 5         |
| 4.5      | Builder Pattern . . . . .                                    | 5         |
| 4.6      | Template Method Pattern . . . . .                            | 6         |
| 4.7      | Dependency Injection — Constructor Injection . . . . .       | 6         |
| 4.8      | Facade Pattern . . . . .                                     | 6         |
| 4.9      | Composite Pattern . . . . .                                  | 6         |
| 4.10     | Service Layer Pattern . . . . .                              | 7         |
| <b>5</b> | <b>Component Interaction &amp; Data Flow</b>                 | <b>7</b>  |
| 5.1      | Request-Response Flow (Backend) . . . . .                    | 7         |
| 5.2      | Frontend Navigation Flow . . . . .                           | 9         |
| 5.3      | Message Format . . . . .                                     | 9         |
| 5.4      | REST API Endpoint Summary . . . . .                          | 9         |
| <b>6</b> | <b>Security in Communication</b>                             | <b>12</b> |
| 6.1      | Current State and Known Gaps . . . . .                       | 12        |
| 6.2      | Authentication — JWT (JSON Web Tokens) . . . . .             | 12        |
| 6.3      | Authorization — Role-Based Access Control (RBAC) . . . . .   | 13        |
| 6.4      | Password Security . . . . .                                  | 13        |
| 6.5      | Data Protection in Transit . . . . .                         | 13        |
| 6.6      | Input Validation . . . . .                                   | 13        |
| 6.7      | CORS Policy . . . . .                                        | 14        |
| 6.8      | Security Flow Summary . . . . .                              | 14        |
| <b>7</b> | <b>Technology Stack Summary</b>                              | <b>15</b> |

## 1. System Overview

The **ViaBalkan Aktourbosna Management System** is a desktop-based tour management application designed for the Balkans region. It enables administrators and tour guides to manage tours, routes, vehicles, guides, expenses, customers, waypoints, and toll information through a rich graphical user interface.

The system is composed of two distinct Maven modules:

| Module       | Technology             | Role                                                          |
|--------------|------------------------|---------------------------------------------------------------|
| tur-frontend | JavaFX 21 (FXML + CSS) | Desktop GUI client running on the user's machine              |
| tur-backend  | Spring Boot 3.2.4      | REST API server providing business logic and data persistence |

Both modules are implemented in **Java 21** and managed as a **Maven multi-module project**. The backend connects to an external **MariaDB** database hosted at `85.98.94.161:27060`.

## 2. Architecture Style

### 2.1. Selected Style: Layered Client-Server Architecture

The system follows a **Client-Server** architecture at the macro level, with each side implementing a **Layered (N-Tier)** structure internally. This combination was selected for the following reasons:

#### 2.1.1. Why Client-Server?

- The JavaFX desktop application and the Spring Boot API are **physically and logically separated**. The backend runs on a server (port 8080); the frontend runs on the client machine.
- Separation allows independent development of the UI and business logic by different team members.
- It enables future scalability: multiple clients could connect to the same backend without any changes to the server.

#### 2.1.2. Why Layered (N-Tier)?

The backend implements **four distinct layers**, each with a single well-defined responsibility:

| # | Layer              | Classes       | Responsibility                                      |
|---|--------------------|---------------|-----------------------------------------------------|
| 1 | Controller Layer   | *Controller   | Parse HTTP requests, return HTTP responses          |
| 2 | DTO + Mapper Layer | *DTO, *Mapper | Transform between API contracts and domain entities |
| 3 | Service Layer      | *Service      | Business logic, transaction management              |
| 4 | Repository Layer   | *Repository   | Data access via Spring Data JPA                     |

- Controllers never touch JPA entities directly — every request and response passes through the **DTO/Mapper layer** first. This is a deliberate architectural boundary, not just a utility.
- Services operate on domain entities internally but always receive and return DTOs at their boundaries with controllers.
- Repositories are the only layer that communicates with the database; no SQL or JPQL appears anywhere else.
- The frontend mirrors the separation with a **Controller** → **ApiService** split: JavaFX controllers call **ApiService** without knowing any HTTP internals.

### 2.1.3. Why NOT Microservices?

This is an academic project with a small team. Microservices would introduce unnecessary infrastructure complexity (service discovery, load balancing, inter-service authentication) without meaningful benefit at this scale.

### 2.1.4. Why NOT Event-Driven?

The system's core operations (CRUD on tours, guides, vehicles) are fundamentally *request-response* in nature. An event-driven approach would overcomplicate simple synchronous operations.

## 3. Architecture Diagram

### 3.1. High-Level System Diagram

The diagram below shows the full architecture of the system from the client machine through the HTTP layer to the backend tiers and database.

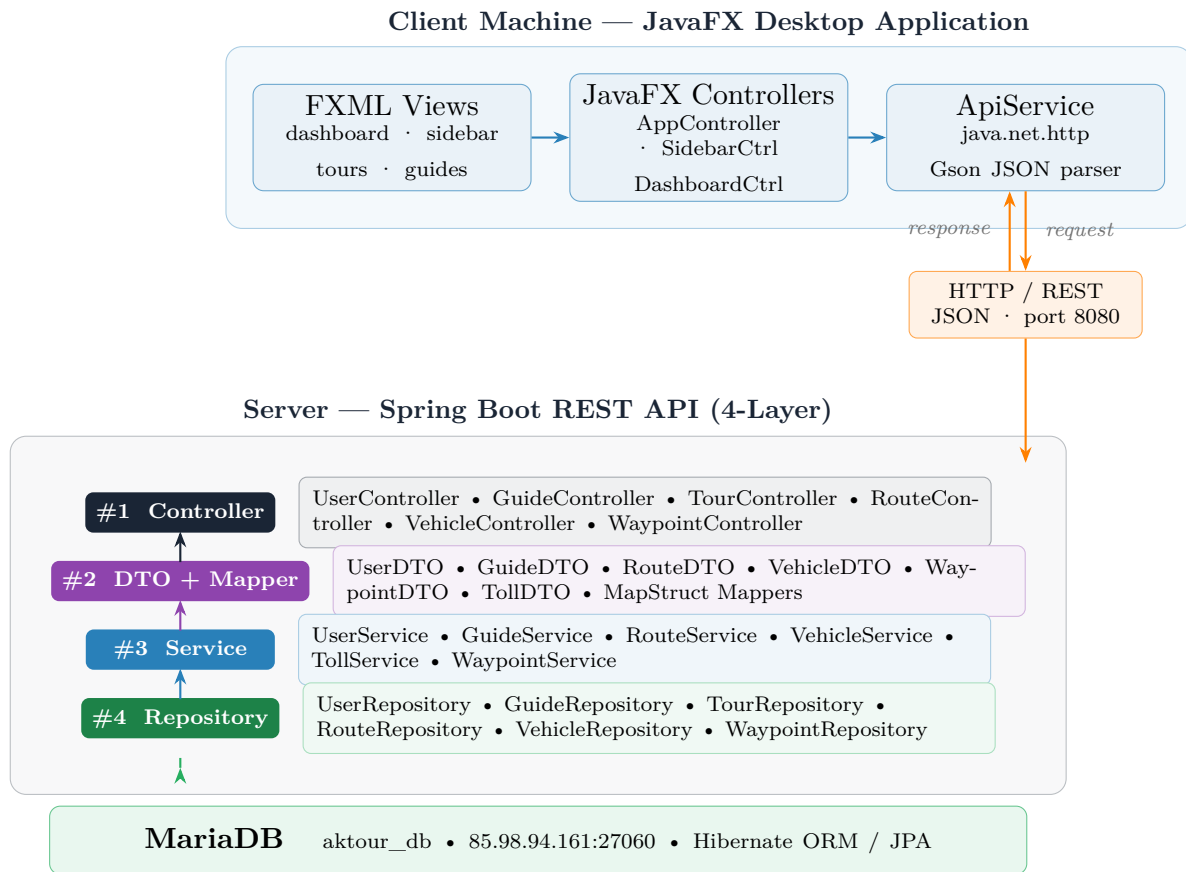


Figure 1: 4-Layer Layered Client-Server Architecture

### 3.2. Data Model Relationships

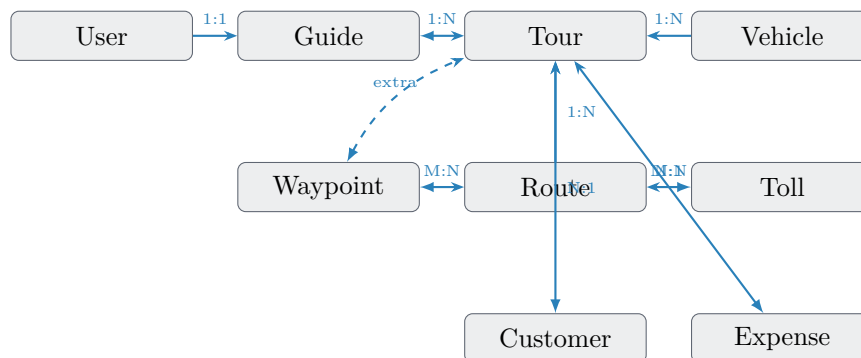


Figure 2: Entity Relationship Overview

## 4. Design Patterns

### 4.1. Repository Pattern

**Where:** org.example.repository — UserRepository, GuideRepository, TourRepository, RouteRepository, VehicleRepository, WaypointRepository, TollRepository

**How:** Each repository interface extends JpaRepository<Entity, Long>, which provides standard CRUD operations without writing SQL.

**Why:** Abstracts all database access behind an interface. The service layer never writes raw queries — it works through the repository contract. This makes it trivial to swap the underlying database technology without touching business logic. The full call chain is: `Controller` → `DTO/Mapper` → `Service` → `Repository` → `Database`.

---

#### 4.2. Data Transfer Object (DTO) Pattern

**Where:** `org.example.application.dto` — `UserRegistrationDTO`, `UserResponseDTO`, `GuideDTO`, `RouteDTO`, `VehicleDTO`, `WaypointDTO`, `TollDTO`

**How:** Controllers receive DTOs in request bodies and return DTOs in responses. JPA entities are never exposed directly to the API layer.

**Why:** Prevents exposing internal entity structure (JPA proxy objects, lazy-loaded collections) to API consumers. It also allows different request and response shapes — `UserRegistrationDTO` includes a password field while `UserResponseDTO` does not, ensuring credentials are never returned in responses.

---

#### 4.3. Mapper Pattern — MapStruct

**Where:** `org.example.application.mapper` — `UserMapper`, `GuideMapper`, `RouteMapper`, `VehicleMapper`, `WaypointMapper`, `TollMapper`

**How:** MapStruct generates type-safe mapping code at compile time. Mappers are Spring components (`componentModel = "spring"`) and injected via constructor injection.

**Why:** Eliminates hand-written, error-prone field-copying code. `GuideMapper` declaratively maps `user.username` → `fullName`; `RouteMapper` composes `WaypointMapper` and `TollMapper` for nested collections.

---

#### 4.4. Singleton Pattern

**Where:** All Spring-managed beans: controllers (`@RestController`), services (`@Service`), repositories (`@Repository`), mappers.

**How:** Spring's IoC container creates one shared instance of each component by default (`scope = singleton`).

**Why:** Services and repositories are stateless and thread-safe by design. A single shared instance reduces memory overhead and prevents inconsistent state across concurrent requests.

---

#### 4.5. Builder Pattern

**Where:** Entity classes — `User`, `Guide`, `Tour`, `Route`, `Vehicle`, etc., annotated with Lombok `@Builder` / `@SuperBuilder`.

**Why:** Entities have many optional fields. For example, `Guide` has `phone`, `baseCity`, `licenseNo`, `dailyFee`, `experience`, `rating`, and `currency`. A builder makes construction readable and avoids telescoping constructors:

```
1 Guide savedGuide = Guide.builder()
2   .user(savedUser)
3   .baseCity("Sarajevo")
4   .dailyFee(150.0)
5   .experience(5)
6   .build();
```

Listing 1: Builder pattern in `GuideService`

---

## 4.6. Template Method Pattern

**Where:** Abstract `User` class in the frontend (`org.example.model`) with concrete subclasses `Admin` and `Guide`.

**How:** The abstract class defines the interface (`login()`, `logout()`, `updateProfile()`) as abstract methods. Concrete subclasses provide role-specific implementations.

**Why:** Enforces that all user types implement the same interface while allowing role-specific behavior. This aligns with the two roles in the system (ADMIN, GUIDE) that have different permissions and views.

---

## 4.7. Dependency Injection — Constructor Injection

**Where:** All Spring controllers and services. Example: `GuideService(GuideRepository, UserRepository, GuideMapper)`.

**Why:** Constructor injection makes dependencies explicit, enables immutability (`final` fields), and simplifies unit testing — classes can be instantiated with mock dependencies without a Spring context.

---

## 4.8. Facade Pattern

**Where:** `ApiService` in the frontend (`org.example.service`).

**How:** `ApiService` hides all HTTP communication details (building requests, sending them, parsing JSON) behind a clean method interface such as `fetchTours()`.

**Why:** Decouples the UI layer from the networking layer. If the HTTP client library or the API URL changes, only `ApiService` needs updating — no controller code changes.

---

## 4.9. Composite Pattern

**Where:** `Tour` entity contains `List<Customer>` and `List<Expense>` with `addCustomer()` and `addExpense()` methods; `Route` contains `List<Waypoint>` and `List<Toll>`.

**Why:** Reflects the real-world domain: a tour is meaningfully composed of customers and expenses; a route is composed of waypoints and tolls. The composite structure keeps related data and behavior together, and JPA cascade operations propagate changes through the aggregate.

---

#### 4.10. Service Layer Pattern

**Where:** `org.example.service` — `UserService`, `GuideService`, `RouteService`, `VehicleService`, `TollService`, `WaypointService`.

**How:** All business logic lives in service classes. Controllers only parse HTTP input and delegate. Repositories only handle persistence.

**Why:** This is the single most important structural pattern in the backend. Without a service layer, controllers would grow to contain business rules — for example, “automatically create a **Guide** record when a **User** with **GUIDE** role registers” — making them impossible to test in isolation or reuse across different endpoints.

## 5. Component Interaction & Data Flow

---

### 5.1. Request-Response Flow (Backend)

A typical API request (creating a new Guide) flows through the layers as follows:

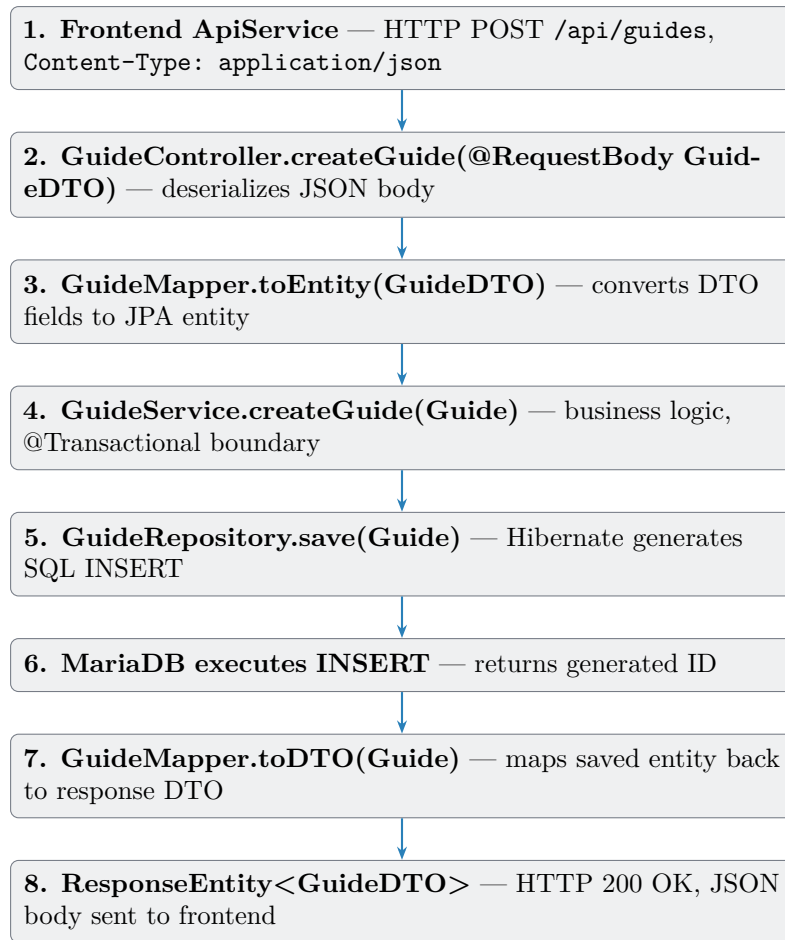


Figure 3: Full request-response lifecycle for Guide creation

## 5.2. Frontend Navigation Flow

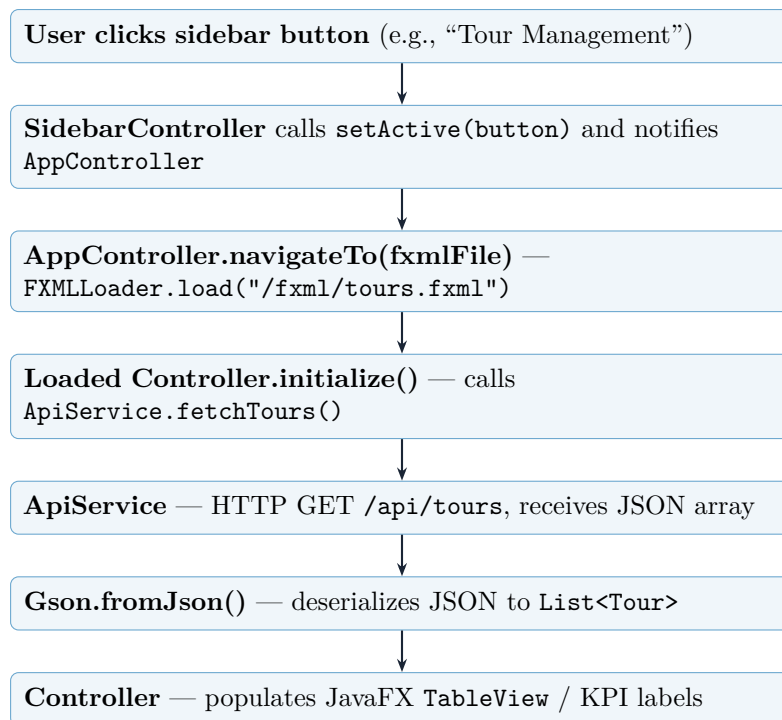


Figure 4: Frontend navigation and data loading flow

## 5.3. Message Format

All API communication uses **JSON** over HTTP:

- **Request:** `application/json` body for POST/PUT; path variables and query parameters for GET/DELETE.
- **Response:** `application/json` — always a DTO, never a raw JPA entity.
- **Serialization:** Spring Boot uses Jackson on the backend; Gson on the frontend.
- **Date format:** ISO 8601 strings (YYYY-MM-DD) for `startDate` and `endDate` fields.

## 5.4. REST API Endpoint Summary

The table below shows the complete intended API surface of the system. Each endpoint is labeled with its current implementation status: **Done** means the endpoint is implemented and tested; **In Progress** means the endpoint is actively being developed at the time of writing.

**Done** implemented    **In Progress** actively being developed

| Users | /api/users |
|-------|------------|
|-------|------------|

| Endpoint        | Method | Purpose             | Status      |
|-----------------|--------|---------------------|-------------|
| /api/users      | GET    | List all users      | Done        |
| /api/users      | POST   | Register new user   | Done        |
| /api/users/{id} | GET    | Get user by ID      | In Progress |
| /api/users/{id} | PUT    | Update user profile | In Progress |
| /api/users/{id} | DELETE | Delete user         | In Progress |

**Guides**

/api/guides

| Endpoint         | Method | Purpose          | Status |
|------------------|--------|------------------|--------|
| /api/guides      | GET    | List all guides  | Done   |
| /api/guides      | POST   | Create new guide | Done   |
| /api/guides/{id} | GET    | Get guide by ID  | Done   |
| /api/guides/{id} | PUT    | Update guide     | Done   |
| /api/guides/{id} | DELETE | Delete guide     | Done   |

**Tours**

/api/tours

| Endpoint        | Method | Purpose         | Status      |
|-----------------|--------|-----------------|-------------|
| /api/tours      | GET    | List all tours  | Done        |
| /api/tours      | POST   | Create new tour | In Progress |
| /api/tours/{id} | GET    | Get tour by ID  | In Progress |
| /api/tours/{id} | PUT    | Update tour     | In Progress |
| /api/tours/{id} | DELETE | Delete tour     | In Progress |

*Tour-Extra Waypoint Association* /api/tours/{tourId}/waypoints/{waypointId}

| Method | Purpose                           | Status      |
|--------|-----------------------------------|-------------|
| POST   | Add extra waypoint to a tour      | In Progress |
| DELETE | Remove extra waypoint from a tour | In Progress |

**Routes**

/api/routes

| Endpoint         | Method | Purpose         | Status      |
|------------------|--------|-----------------|-------------|
| /api/routes      | GET    | List all routes | Done        |
| /api/routes      | POST   | Create route    | Done        |
| /api/routes/{id} | GET    | Get route by ID | Done        |
| /api/routes/{id} | PUT    | Update route    | In Progress |
| /api/routes/{id} | DELETE | Delete route    | In Progress |

*Route–Waypoint Association**/api/routes/{rId}/waypoints/{wId}*

| Method | Purpose                        | Status |
|--------|--------------------------------|--------|
| POST   | Link a waypoint to a route     | Done   |
| DELETE | Unlink a waypoint from a route | Done   |

**Vehicles***/api/vehicles*

| Endpoint           | Method | Purpose           | Status |
|--------------------|--------|-------------------|--------|
| /api/vehicles      | GET    | List all vehicles | Done   |
| /api/vehicles      | POST   | Create vehicle    | Done   |
| /api/vehicles/{id} | GET    | Get vehicle by ID | Done   |
| /api/vehicles/{id} | PUT    | Update vehicle    | Done   |
| /api/vehicles/{id} | DELETE | Delete vehicle    | Done   |

**Waypoints***/api/waypoints*

| Endpoint            | Method | Purpose            | Status      |
|---------------------|--------|--------------------|-------------|
| /api/waypoints      | GET    | List all waypoints | Done        |
| /api/waypoints      | POST   | Create waypoint    | Done        |
| /api/waypoints/{id} | GET    | Get waypoint by ID | In Progress |
| /api/waypoints/{id} | PUT    | Update waypoint    | In Progress |
| /api/waypoints/{id} | DELETE | Delete waypoint    | In Progress |

**Expenses***/api/tours/{tourId}/expenses*

| Endpoint                          | Method | Purpose                 | Status      |
|-----------------------------------|--------|-------------------------|-------------|
| /api/tours/{tourId}/expenses      | GET    | List expenses of a tour | In Progress |
| /api/tours/{tourId}/expenses      | POST   | Add expense to a tour   | In Progress |
| /api/tours/{tourId}/expenses/{id} | GET    | Get expense by ID       | In Progress |
| /api/tours/{tourId}/expenses/{id} | PUT    | Update expense          | In Progress |
| /api/tours/{tourId}/expenses/{id} | DELETE | Delete expense          | In Progress |

**Other**

| Endpoint  | Method | Purpose                    | Status |
|-----------|--------|----------------------------|--------|
| /swagger  | GET    | OpenAPI / Swagger UI       | Done   |
| /api-docs | GET    | OpenAPI JSON specification | Done   |

## 6. Security in Communication

### Note

This section describes both the **current state** and the **intended security design**. The current codebase operates without an authentication framework (early development phase); the items marked as “planned” are the target architecture for the next development iteration.

### 6.1. Current State and Known Gaps

The following security concerns exist in the current prototype:

- No Spring Security dependency; all API endpoints are publicly accessible.
- Passwords transmitted and potentially stored as plaintext.
- `@CrossOrigin(origins="*")` on `TourController` permits requests from any origin.
- Database credentials are hardcoded in `application.properties`.
- No HTTPS enforced; communication travels over plain HTTP.

### 6.2. Authentication — JWT (JSON Web Tokens)

**Goal:** Only users who possess a valid, server-issued JWT token can send authenticated requests.

**Login flow:**

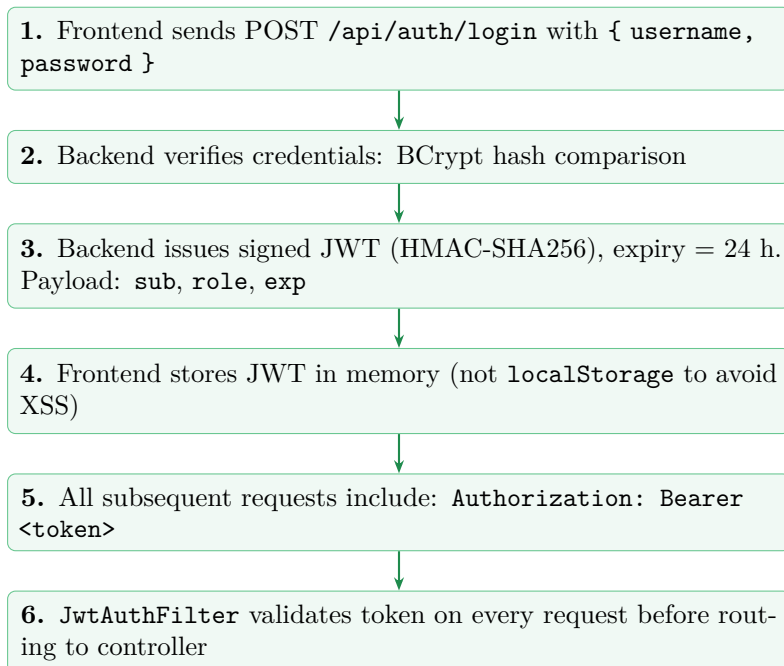


Figure 5: JWT authentication flow

### 6.3. Authorization — Role-Based Access Control (RBAC)

Access control is enforced at method level using Spring Security's `@PreAuthorize`. The system has two roles: **ADMIN** and **GUIDE**.

| Operation                          | ADMIN | GUIDE            |
|------------------------------------|-------|------------------|
| View all tours / routes / vehicles | ✓     | ✓                |
| Create / edit / delete tours       | ✓     | ×                |
| Manage guides                      | ✓     | Own profile only |
| Create / delete vehicles           | ✓     | ×                |
| View expense reports               | ✓     | Own tours only   |
| Register new users                 | ✓     | ×                |

```

1 @PreAuthorize("hasRole('ADMIN')")
2 @DeleteMapping("/{id}")
3 public ResponseEntity<Void> deleteGuide(@PathVariable Long id
4     ) {
5     guideService.deleteGuide(id);
6     return ResponseEntity.noContent().build();
7 }

```

Listing 2: RBAC enforcement with `@PreAuthorize`

### 6.4. Password Security

Passwords are hashed using **BCrypt** (Spring Security's `BCryptPasswordEncoder`, cost factor 12) before storage. Raw passwords are never stored, logged, or returned in any response. The `UserResponseDTO` deliberately omits the password field.

### 6.5. Data Protection in Transit

- **HTTPS (TLS 1.2+)**: In production, the backend server is deployed behind HTTPS. All HTTP traffic between the JavaFX client and the server is encrypted via TLS.
- **JWT integrity**: Tokens are signed with HMAC-SHA256. A tampered token fails signature verification and is immediately rejected with HTTP 401.
- **Database connection**: The MariaDB connection string will include `useSSL=true&requireSSL=true` to encrypt data in transit between the Spring Boot server and the database.

### 6.6. Input Validation

DTOs are annotated with Bean Validation constraints to reject malformed input before it reaches the service layer:

```

1 public class UserRegistrationDTO {
2     @NotBlank
3     @Size(min = 3, max = 50)

```

```

4     private String username;
5
6     @Email @NotBlank
7     private String email;
8
9     @NotBlank
10    @Size(min = 8)
11    private String password;
12 }

```

Listing 3: Bean Validation on UserRegistrationDTO

Validation is triggered via `@Valid` on controller parameters. On violation, Spring Boot returns HTTP 400 with a structured error body — the malformed request never reaches the service layer.

## 6.7. CORS Policy

Since the client is a JavaFX desktop application (not a browser), CORS headers are less critical than for web frontends. However, the current `@CrossOrigin(origins="*")` will be replaced with a restrictive configuration in production:

```

1 @Bean
2 public CorsConfigurationSource corsConfigurationSource() {
3     CorsConfiguration config = new CorsConfiguration();
4     config.setAllowedOrigins(List.of("http://localhost:*"));
5     config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
6     config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
7     return new UrlBasedCorsConfigurationSource();
8 }

```

Listing 4: Restrictive CORS configuration (planned)

## 6.8. Security Flow Summary

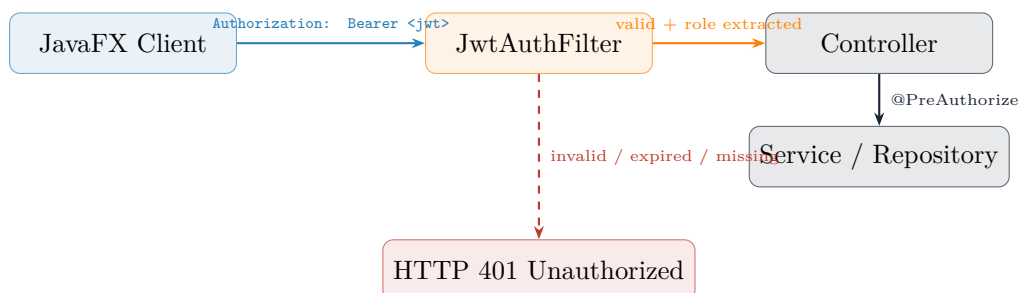


Figure 6: Security enforcement flow on every API request

## 7. Technology Stack Summary

| Component          | Technology                  | Version          | Purpose                        |
|--------------------|-----------------------------|------------------|--------------------------------|
| Backend Framework  | Spring Boot                 | 3.2.4            | REST API server                |
| Frontend Framework | JavaFX                      | 21.0.2           | Desktop GUI                    |
| Language           | Java                        | 21               | Both modules                   |
| ORM                | Spring Data JPA + Hibernate | 6.x              | Database access                |
| Database           | MariaDB / MySQL             | 5.7+             | Persistent storage             |
| Object Mapping     | MapStruct                   | 1.5.5            | DTO ↔ Entity conversion        |
| Boilerplate        | Lombok                      | 1.18.30          | Builders, getters, setters     |
| API Docs           | SpringDoc OpenAPI           | 2.x              | Swagger UI at <b>/swagger</b>  |
| JSON (frontend)    | Gson                        | 2.10.1           | HTTP response deserialization  |
| HTTP Client        | <code>java.net.http</code>  | Java 21 built-in | Frontend → Backend calls       |
| Build Tool         | Maven                       | 3.x              | Dependency management          |
| Security (planned) | Spring Security + JWT       | 6.x              | Authentication & authorization |